Suivi des phases du projet Ndadiée

État du dossier au 6 novembre 2026

Ce document est un aide-mémoire de travail fondé sur les demandes, les fichiers du dépôt et les résultats consignés dans les audits. Il sépare le travail réalisé des points encore à valider. Il ne constitue pas une preuve d'hébergement public ni de conformité officielle au concours.

1. Résumé exécutif

Ndadiée dispose d'une documentation structurée sur le produit, son architecture, ses fonctionnalités, ses API, son développement, son déploiement, la qualité/sécurité et le concours CADEV. La configuration backend a ensuite été durcie : clé JWT obligatoire, seeders désactivés par défaut et gestion de schéma par Flyway avec validation Hibernate.

La suite backend a été exécutée contre PostgreSQL 17 temporaire : 313 tests, 0 échec, 0 erreur, 0 ignoré. Les commandes frontend npm run build, npm run lint et npm run i18n:check ont réussi selon l'audit opérationnel.

Ces résultats ne valident pas les parcours métier complets. Aucun parcours d'inscription, projet, candidature, forum, collaboration, revue, messagerie ou administration n'a été exécuté dans le dernier audit fonctionnel. La production n'est pas déclarée prête : compatibilité de la base réelle, déploiement public, intégrations externes et preuves d'exploitation restent à établir.

2. Journal des phases

1 — Documentation fonctionnelle

Travail réalisé et traces: Documentation des fonctionnalités, règles métier, rôles, parcours et limites dans docs/03-fonctionnalites/.

État / limite: Documentée à partir du dépôt ; les parcours ne sont pas tous validés par exécution.

2 — Documentation API

Travail réalisé et traces: Contrôleurs REST, DTO, sécurité, erreurs et WebSocket décrits dans docs/04-api/.

État / limite: À rapprocher du code en cas d'évolution des endpoints.

3 — Manuel utilisateur

Travail réalisé et traces: Guides simplifiés destinés aux utilisateurs demandés.

État / limite: Non terminé dans le dépôt : le bilan documentaire indique que docs/05-guides-utilisateur/ ne contient pas les guides attendus.

4 — Guide développeur

Travail réalisé et traces: Prérequis, installation, configuration, structure, démarrage, base de données, tests et contribution dans docs/06-developpement/.

État / limite: Documentation présente ; commandes liées au dépôt, à revalider si les scripts changent.

5 — Déploiement et exploitation

Travail réalisé et traces: Architecture proposée, Docker, configuration, PostgreSQL, stockage, sauvegardes, supervision et procédure de mise en production dans docs/07-deploiement/.

État / limite: Pas de preuve d'hébergement Datacloud ni de déploiement public. Le Compose présent est limité à PostgreSQL local ; aucun déploiement applicatif n'a été effectué.

6 — Qualité et sécurité

Travail réalisé et traces: Stratégie de tests, revue des contrôles, risques, secrets, CORS et recommandations dans docs/08-qualite-securite/.

État / limite: Revue statique ciblée ; elle ne vaut pas audit exhaustif ni certification.

7 — Conformité CADEV

Travail réalisé et traces: Présentation du thème, matrices et checklists dans docs/09-conformite-cadev/.

État / limite: Le cadre vient des informations communiquées. Composition de l'équipe, contributions, usage historique d'IA, licences complètes et URL de production non établis.

8 — Finalisation documentaire

Travail réalisé et traces: README, index, bilan, plan d'actions, checklist et scénario de démonstration ; vérifications de liens et de chemins source consignées.

État / limite: Le scénario ne doit être joué que pour les fonctions réellement disponibles.

Audit opérationnel

Travail réalisé et traces: Contrôles de démarrage et de configuration, tests backend/frontend, examen des artefacts de déploiement et parcours métier.

État / limite: Le dernier état est consigné dans docs/08-qualite-securite/audit-operationnel.md et docs/07-deploiement/etat-preparation-production.md. Tests automatiques passés, mais parcours end-to-end et production non vérifiés.

Stabilisation sécurité et base

Travail réalisé et traces: Clé JWT sans valeur de repli, validation au démarrage, seeders opt-in, Flyway V1 et Hibernate validate. Migration testée sur PostgreSQL temporaire neuf ; baseline explicite testée sur schéma historique jetable.

État / limite: Base réelle non comparée à V1. Ne pas baseliner une base durable sans sauvegarde, comparaison et procédure contrôlée.

Audit fonctionnel approfondi

Travail réalisé et traces: Correspondances frontend/backend, services, repositories, entités et tests analysées dans audit-fonctionnel.md et matrice-fonctionnelle.md.

État / limite: Audit du code, pas une validation de parcours en fonctionnement.

Les demandes initiales ont parfois employé le même numéro de phase pour des sujets différents. Le journal ci-dessus nomme donc les travaux plutôt que de prétendre à une numérotation officielle exhaustive.

3. Changements techniques de stabilisation

• JWT : le secret n'a plus de fallback de développement ; le démarrage refuse une valeur absente, mal formée ou inférieure à 256 bits.

• Données de démonstration : désactivées par défaut ; l'activation doit être explicite et exige les mots de passe configurés.

• Base : Flyway est activé avec la migration initiale V1 ; Hibernate utilise validate pour ne pas créer, mettre à jour ou supprimer silencieusement le schéma.

• Base neuve : V1 et la validation Hibernate ont été testées sur un PostgreSQL temporaire.

• Schéma historique jetable : un essai de baseline Flyway version 1 et de validation Hibernate a été réalisé sans modifier les nombres relevés de lignes de test. Cela ne démontre pas la compatibilité avec une base de production.

• Préservation : le volume PostgreSQL préexistant du projet n'a pas été utilisé, modifié ou supprimé pendant les tests décrits.

Références : backend/src/main/resources/application.yaml, application-local.yaml, application-prod.yaml, backend/src/main/resources/db/migration/V1__initial_schema.sql, backend/src/main/java/com/hautetech/ndadiee/security/JwtService.java.

4. Résultats vérifiés et portée

Backend

Résultat consigné: 313 tests, 0 échec, 0 erreur, 0 ignoré, PostgreSQL 17 temporaire sans volume.

Ce que cela prouve / ne prouve pas: La suite automatisée backend réussit dans cet environnement ; elle ne valide pas tous les parcours réels.

Frontend

Résultat consigné: npm run build, npm run lint, npm run i18n:check réussis.

Ce que cela prouve / ne prouve pas: Build, lint et cohérence i18n ; pas la qualité de tous les parcours UI/API.

Migration

Résultat consigné: V1 appliquée sur base neuve ; baseline testée sur schéma historique jetable.

Ce que cela prouve / ne prouve pas: Comportement contrôlé sur les bases temporaires ; aucune conclusion sur une base distante.

Navigation locale

Résultat consigné: L'accueil frontend a répondu en HTTP 200 lors de l'audit opérationnel antérieur.

Ce que cela prouve / ne prouve pas: Seulement un smoke test d'accueil, pas une démo complète.

Parcours fonctionnels

Résultat consigné: Non exécutés dans l'audit fonctionnel approfondi.

Ce que cela prouve / ne prouve pas: Tous les parcours restent à valider en staging avec données synthétiques.

Production et CADEV

Résultat consigné: Aucun déploiement public ni URL de production vérifié.

Ce que cela prouve / ne prouve pas: Il faut fournir des preuves indépendantes : URL, accès externe, captures et tests datés.

L'état détaillé et daté reste dans l'audit opérationnel et l'état de préparation à la production.

5. Constats fonctionnels à retenir

L'audit fonctionnel approfondi et la matrice fonctionnelle confirment la présence dans le code des modules de profils, projets/candidatures, communauté, hackathons, workspace, reviews, messagerie, notifications, GitHub et administration. Les constats suivants sont des écarts de code/interface, non des résultats de test utilisateur :

• Aucun flux de récupération de mot de passe ni OAuth GitHub/Google/GitLab n'a été trouvé.

• La déconnexion supprime les jetons côté navigateur ; aucun endpoint de révocation serveur n'a été trouvé.

• L'édition et la suppression de projet sont présentes côté backend/service, mais aucun écran ou appel de mutation frontend utilisé n'a été trouvé.

• La gestion CRUD des hackathons est côté backend ; aucune interface de modération correspondante n'a été trouvée.

• Des hooks/services d'articles existent sans page/route dédiée identifiée.

• Les revues et commentaires de code sont conservés par Ndadiée ; leur publication vers GitHub n'a pas été trouvée.

• Le profil n'a pas de chronologie d'expériences professionnelles ni de portfolio autonome identifié.

• La modération admin est limitée aux questions, réponses et projets dans le type de contenu contrôlé.

• Aucun test frontend de composant/parcours n'a été identifié lors de l'inventaire.

6. Recommandations avant d'avancer

Priorité 1 — Valider les parcours

Préparer un environnement de staging jetable, avec deux comptes développeur synthétiques et un compte de modération dédié. Exécuter et noter : inscription/connexion, profil, création et recherche de projet, candidature/décision, sujet/réponse/commentaire, tâches et sprint, revue, conversation entre deux comptes, accusé de lecture, notifications et administration.

Pour chaque parcours, conserver le commit, environnement, préconditions, étapes, résultat observé, erreurs et preuve expurgée. Un test unitaire ou la présence d'un écran ne suffit pas à déclarer le parcours validé.

Priorité 2 — Sécuriser toute base durable

Avant une baseline Flyway sur la base cible : sauvegarde vérifiée, comparaison du schéma avec V1, validation de la stratégie de retour arrière et accord explicite de l'opérateur. Aucun test de cette session ne justifie une baseline automatique sur une base existante.

Priorité 3 — Préparer l'exploitation et la candidature

• Prouver l'injection des secrets, le profil actif, SEED_ENABLED=false, la persistance des médias, SMTP/GitHub, HTTPS/CORS, sauvegarde/restauration, logs et surveillance.

• Collecter une URL publique si elle existe, vérifier l'accès depuis un réseau externe et prendre des captures datées. Ne pas annoncer Datacloud sans preuve.

• Confirmer le roster, les contributions, les licences et la déclaration d'usage IA avec l'équipe et le règlement effectif.

• Produire les guides utilisateur manquants, ou réduire explicitement leur périmètre si la bêta ne les exige pas.

• Garder visibles les fonctions non disponibles (récupération de compte, OAuth, interfaces d'administration d'événements/articles) au lieu de les présenter comme opérationnelles.

7. Ordre de lecture conseillé

1. Index documentaire
2. Bilan documentaire
3. Plan des actions restantes
4. Audit fonctionnel et matrice fonctionnelle
5. Audit opérationnel
6. État de préparation à la production
7. Guides fonctionnels, API, développeur et conformité dans les sections 03 à 09

8. Décision de passage

Possible : poursuivre la préparation et les essais sur un staging isolé, sans données réelles sensibles.

Non justifié à ce stade : déclarer la bêta validée, annoncer un hébergement public/Datacloud, appliquer une baseline sur la base réelle ou lancer une production sans vérification de compatibilité, parcours métier et procédures opérationnelles.

Ce journal peut évoluer après chaque validation : ajouter la date, le commit concerné, la preuve et le résultat ; ne pas remplacer un état « non vérifié » par « réussi » sans nouvel essai documenté.